Geometry Modeling (CM1.1)

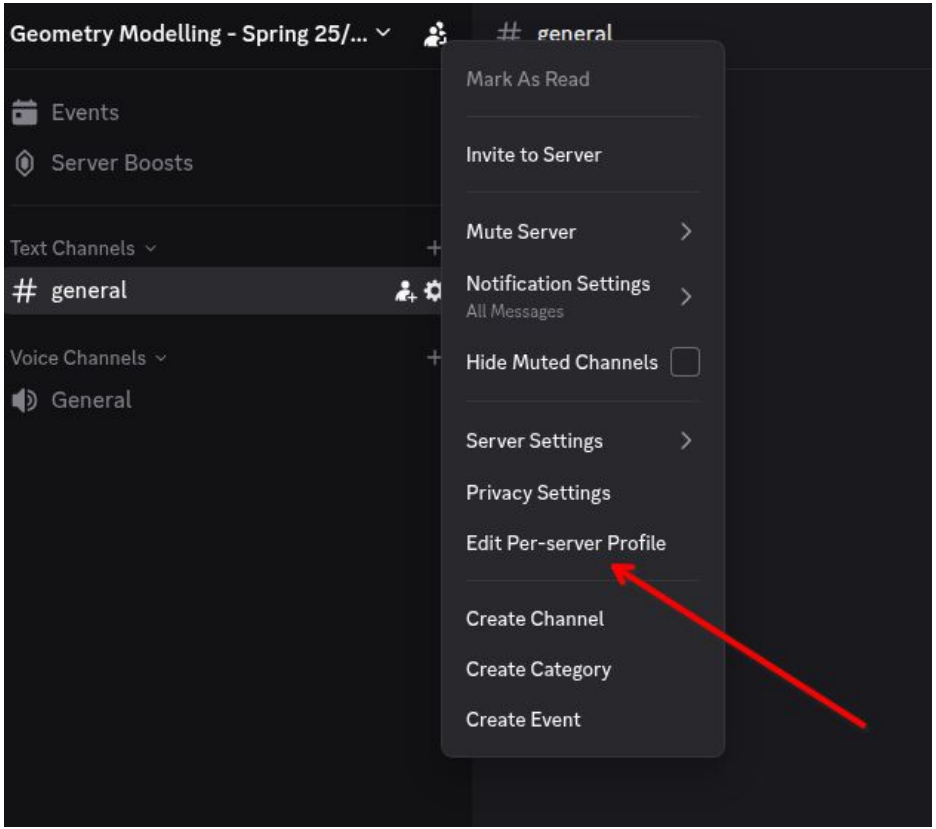
Kacper PLUTA (fr. Casper Plouta) (kacper.pluta@esiee.fr)

Organisation

Participation obligatoire à Discord :

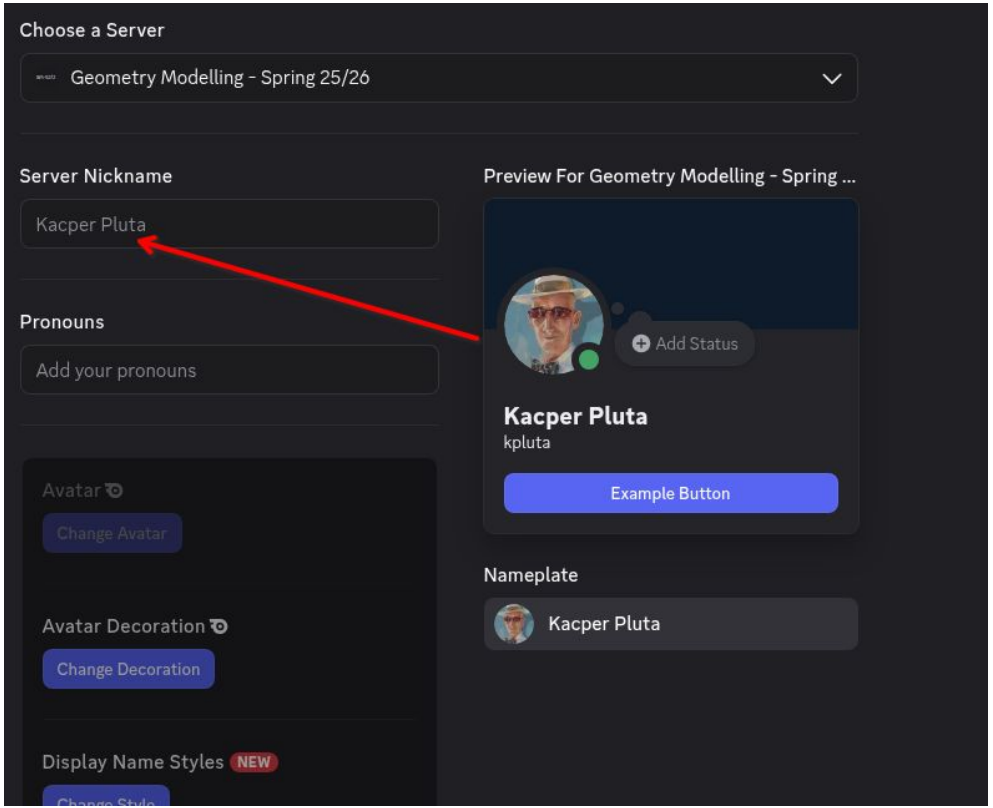
https://discord.gg/QDRJgUFQ

Obligatory configuration



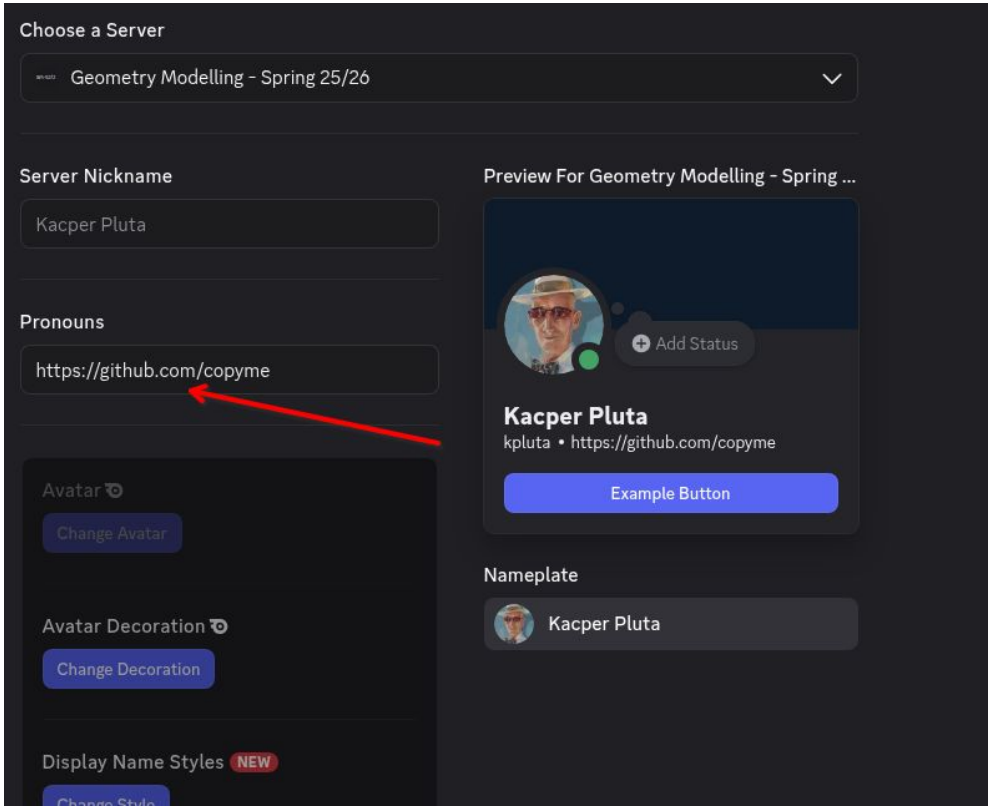
Right-click on the server name and then select “Edit Per-server Profile”

Obligatory configuration



Set your “Server Nickname” to your real name

Obligatory configuration



Set the field “Pronouns” to the address of your PRIVATE GitHub repository used for the course. Add user “copyme” as a collaborator!

Configuration obligatoire

Commits for practical assignments (TP) on a Git repository are mandatory.

Commits must be made regularly. Repositories submitted all at once in a single push will be rejected and will receive a score of zero.

Each repository must include a folder containing screenshots or videos documenting the completed tasks. These screenshots must also be posted on Discord in the #progress channel.

Evaluation

60% exam and 40% practical work (assessment of the work completed during the course, also based on the frequency and quality of commits).

In addition, any undocumented use of AI tools, including Copilot in Visual Studio, will be strictly penalized.

Homework

Windows: Install Visual Studio (not Visual Studio Code) with the Desktop Development with C++ workload.

Linux: For now, nothing to do.

macOS: Pray that the system API has not changed again…

Winter 2025: Objectives and Expectations

Course Objectives

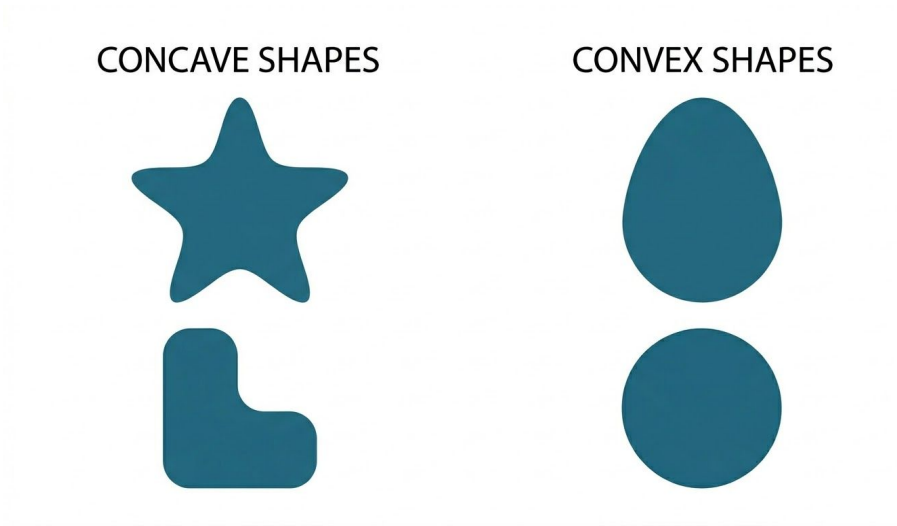
- Introduce common representations of geometric data and the principles guiding their use.
- Develop proficiency in working with 3D geometric data through appropriate data structures.
- Build foundational skills in implementing core geometry-processing algorithms
- Advance students' understanding of more complex algorithms, such as mesh simplification and subdivision.
- Foster the ability to design and develop original geometry-processing algorithms for advanced or creative applications.

Winter 2025: Objectives and Expectations

By the end of this course, students will be able to:

- Explain common representations of geometric data and the reasoning behind their use.
- Work effectively with 3D geometric data through appropriate data structures.
- Implement foundational geometry-processing algorithms, including normal vector computation, ear-clipping triangulation, and related operations.
- Analyze and implement more advanced algorithms, such as mesh simplification and subdivision.
- Design and develop original geometry-processing algorithms for advanced or creative applications.

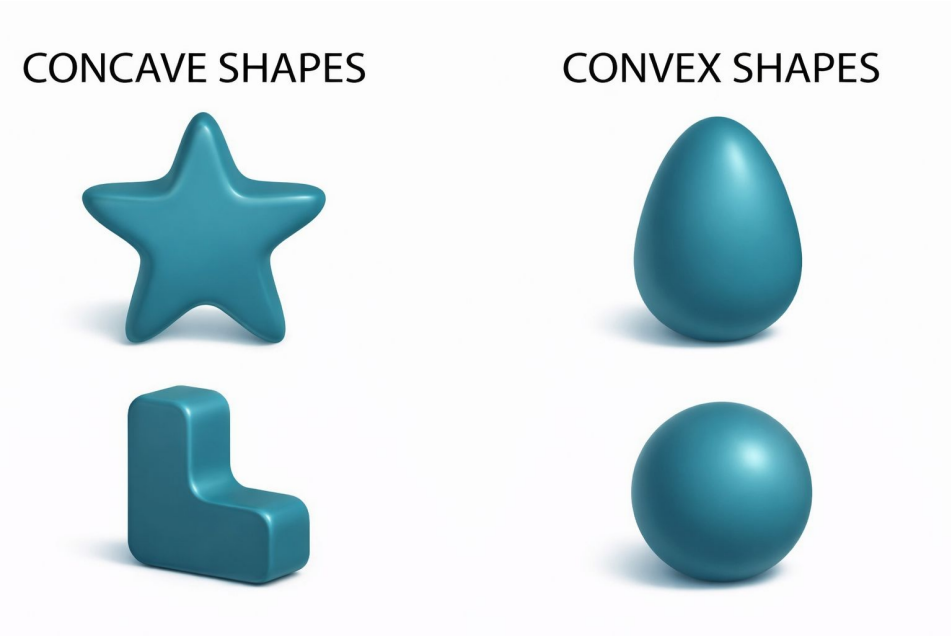
Exercise 1a: 2D warmup



How would you represent these shapes in a machine?

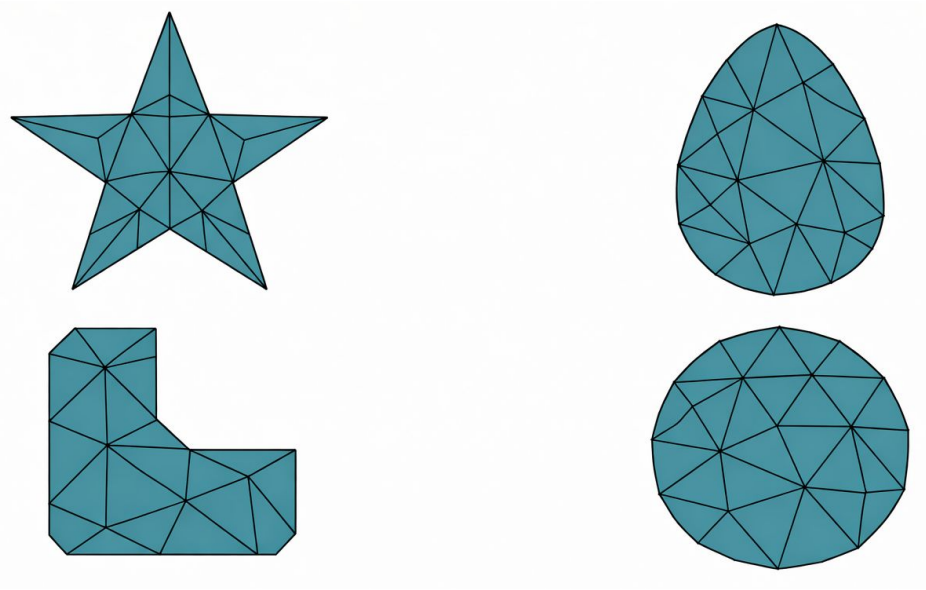
Is it possible to represent smooth geometries natively, or is discretization required? If needed why? Can you propose a discretization scheme? Propose a data model capable of storing such shapes.

Exercise 1b: 3D extension



Could this model be extended to three dimensions? Please provide the necessary adjustments and a demonstration of the 3D implementation.

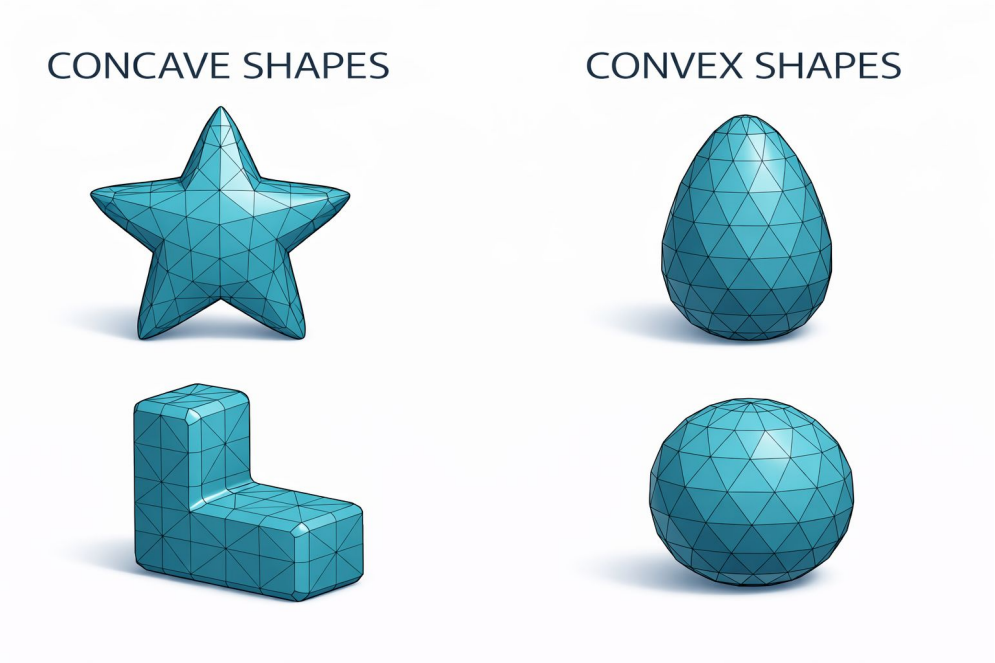
Exercise 2a: polyhedral meshes



Can your solution (data structure) be used to store polyhedral 2D shapes?

How would you store such a geometric data in a file?

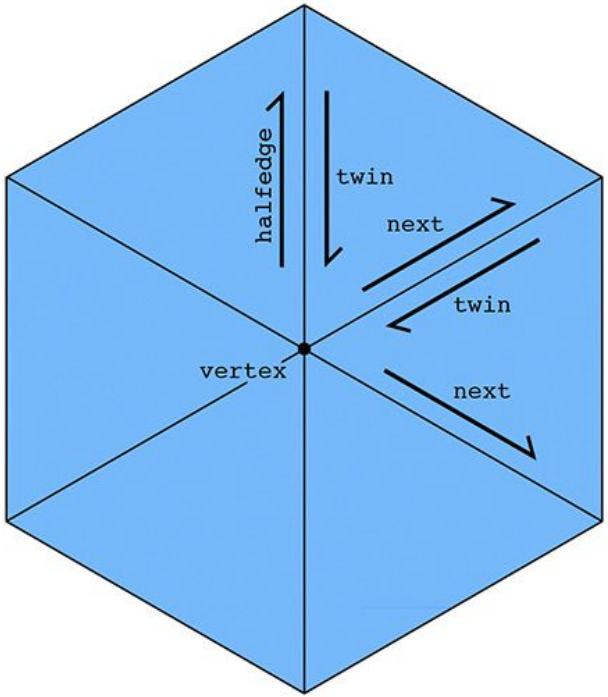
Exercise 2b: polyhedral meshes



Can you extend your solution from the previous slide to 3D?

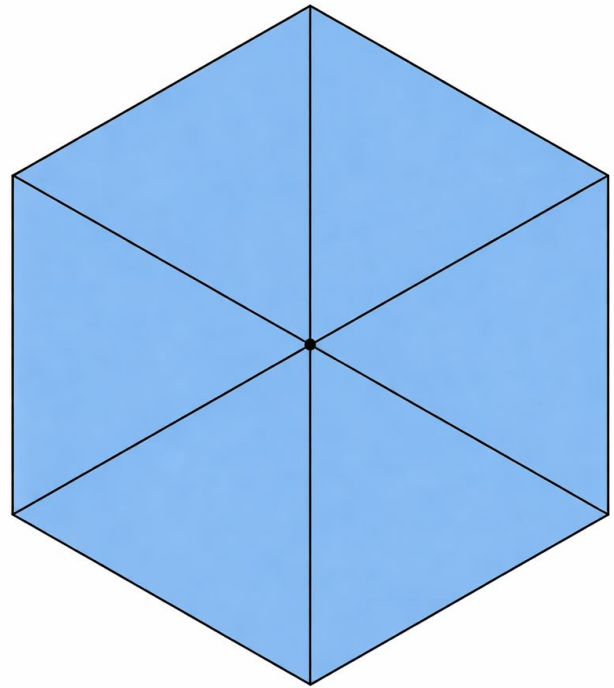
3D Geometry: Representation - Half-edge data structure

Mesh element	Pointers
Vertex	one halfedge
Face	one halfedge
Halfedge	twin, next, vertex, face

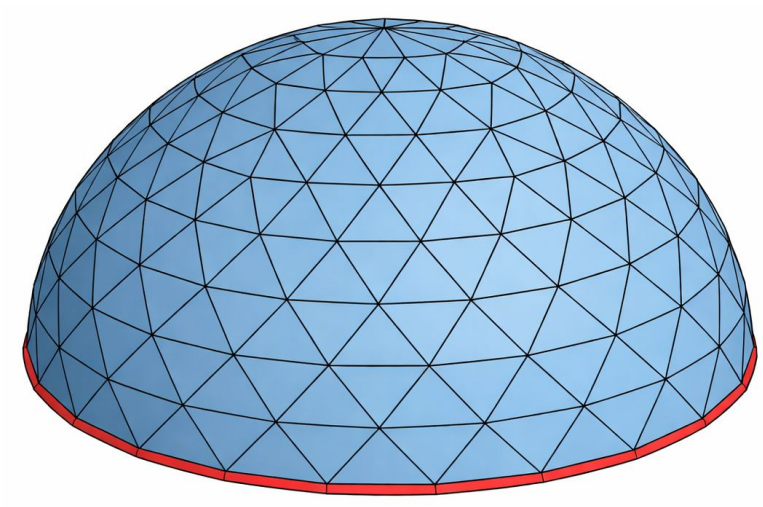


Exercise 3: Data structures and algorithms

Propose an algorithm for traveling around the vertex. In using our data structure provide an illustration for each step of the algorithm.

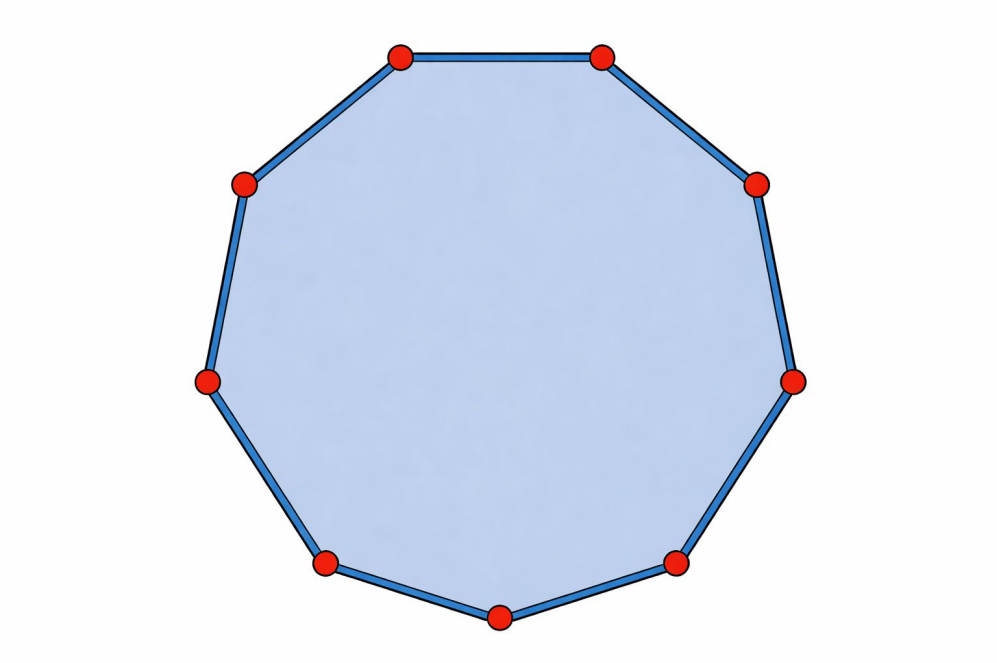


Exercise 4: Data structures and algorithms



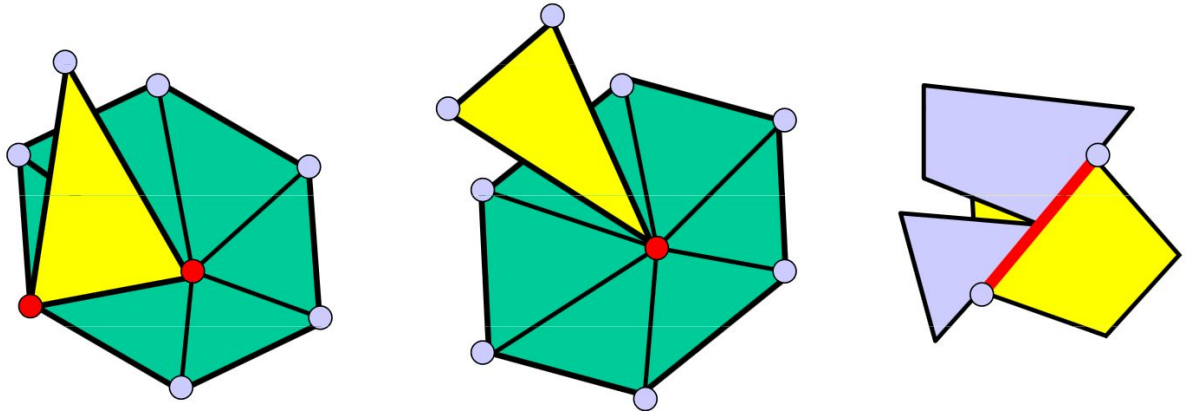
Using your data model propose an algorithm to traverse only the boundary edges.

Exercise 4: Data structures and algorithms



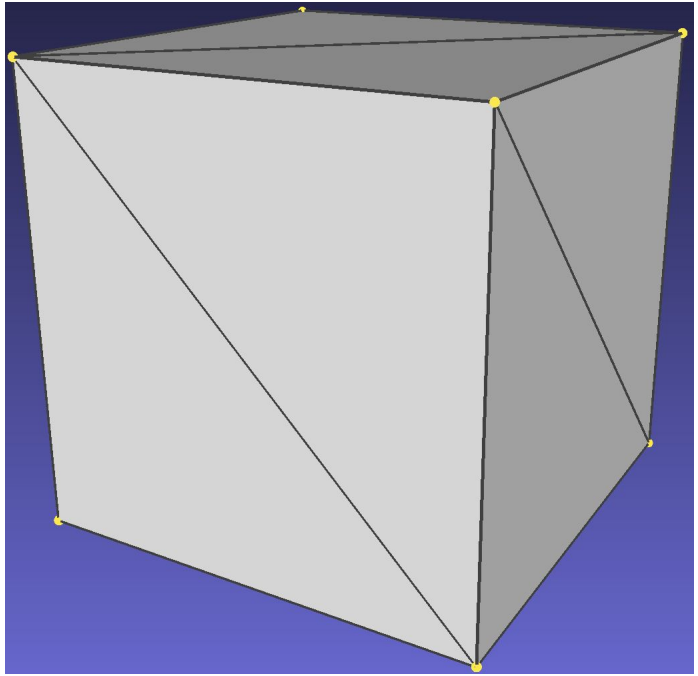
Using your data model propose an algorithm to triangulate convex polygons.

Exercise 5: Non-manifold meshes



The configurations shown above are non-manifold. Can our data model represent them? If not, do you consider this a significant limitation?

3D Geometry Representation: PLY file format



```
ply
format ascii 1.0
element vertex 8
property float x
property float y
property float z
element face 12
property list uchar uint vertex_indices
end_header
-1.000000 -1.000000 1.000000
-1.000000 1.000000 1.000000
-1.000000 1.000000 -1.000000
1.000000 1.000000 1.000000
1.000000 1.000000 -1.000000
1.000000 -1.000000 1.000000
1.000000 -1.000000 -1.000000
-1.000000 -1.000000 -1.000000
3 0 1 2
3 1 3 4
3 3 5 6
3 0 7 5
3 7 2 4
3 5 3 1
3 7 0 2
3 2 1 4
3 4 3 6
3 5 7 6
3 6 7 4
3 0 5 1
```

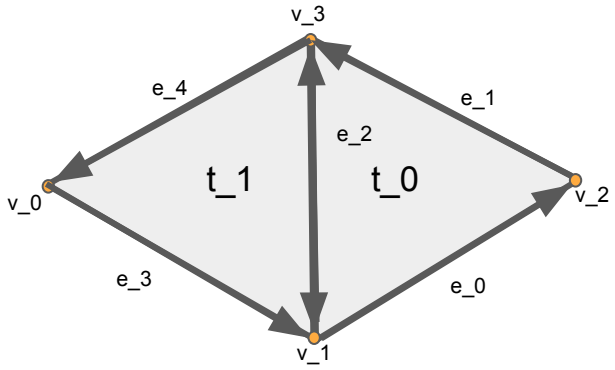
header block

list of vertices (counted from 0)

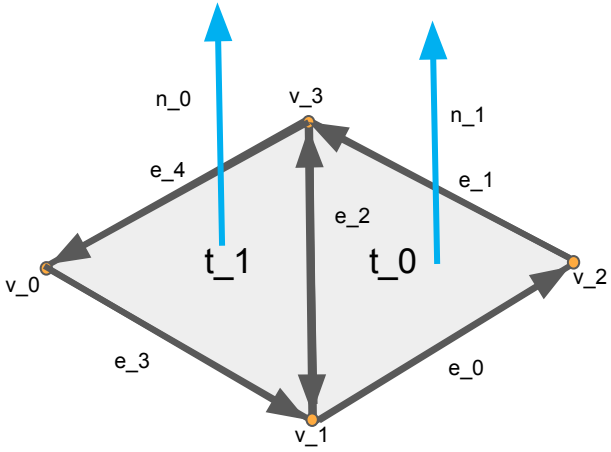
list of faces

3D Geometry: Representation

```
ply
format ascii 1.0
element vertex 4
property float x
property float y
property float z
element face 2
property list uchar uint vertex_indices
end_header
-1.0 -1.00 0.0
-1.0 1.0 0.0
-1.01.0 0.0
1.0 1.0 0.0
3 1 2 3
3 0 1 3
```



Question: Why the consistent orientation of edges is important?

3D Geometry: Representation

```
ply
format ascii 1.0
element vertex 4
property float x
property float y
property float z
element face 2
property list uchar uint
vertex_indices
end_header
-1.0 -1.00 0.0
-1.0 1.0 0.0
-1.01.0 0.0
1.0 1.0 0.0
3 1 2 3
3 0 1 3
```